Protocolo DNS

Resolución de nombres en Internet

📧 José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📚 SRI · Servicios de Red e Internet

01

Conceptos sobre DNS

Nombres, zonas y registros

¿Qué es DNS?

> *"**Domain Name System** es el protocolo que permite **guardar y consultar información** asociada a un nombre de dominio: dirección IP de un equipo, nombre asociado a una IP, servidor de correo de un dominio...*

IDEAS CLAVE

- Las redes funcionan con **direcciones IP**, las personas con **nombres**
- DNS es el **directorio jerárquico** que traduce de uno a otro a escala global
- Es **distribuido**: ningún servidor conoce todos los nombres
- Su funcionamiento se basa en **delegación** entre dominios

ℹ️ El servicio escucha por defecto en el puerto **53**, sobre **UDP** (consultas) y **TCP** (transferencias y respuestas grandes).

Tipos de nombres

HOSTNAME

- **Una sola palabra** que identifica al equipo
- Se guarda en `/etc/hostname`

FQDN — *FULLY QUALIFIED DOMAIN NAME*

- Hostname **+ dominio completo**: `macaco.gonzalonazareno.org.`
- Identifica al equipo de forma **inequívoca**

DOMAIN NAME

- Sucesión de nombres separados por **puntos**
- Ejemplo: `gonzalonazareno.org`

DOMINIO RAÍZ

- Es el **punto final**: todo nombre acaba en `.`

TLD — *TOP LEVEL DOMAIN*

- Dominios bajo el raíz: `com`, `org`, `es`...

Zonas DNS

> *Una **zona** agrupa la información de un nombre de dominio. El servidor que guarda una zona es su **autoridad**.*

RESOLUCIÓN DIRECTA

- Conjunto de nombres bajo un dominio
- Permite obtener la **IP** a partir del **nombre**
- En `gonzalonazareno.org`: `macaco`, `www`, `dit`, `openstack`...

RESOLUCIÓN INVERSA

- Permite obtener el **nombre** a partir de la **IP**
- Depende del **direccionamiento** de red
- Útil para registros de correo, *logs* y comprobaciones

ℹ️ El **fichero de zona** es un archivo de texto donde se guardan los **registros** (cada tipo de información tiene su propio tipo de registro).

Resolución inversa: `in-addr.arpa`

¿Cómo se busca un **nombre a partir de una IP** si DNS indexa por nombres? Se convierte la IP en un nombre bajo el dominio `in-addr.arpa`:

- Se escriben los **octetos al revés** y se añade `.in-addr.arpa.`
- Se hace al revés porque en un nombre lo **más general va a la derecha**, y en una IP lo más general (la red) va a la izquierda

```
IP:      8.8.8.8
Nombre:  8.8.8.8.in-addr.arpa.     →  PTR  →  dns.google.

IP:      172.22.0.1
Nombre:  1.0.22.172.in-addr.arpa.  →  PTR  →  (nombre del equipo)
```

- El tipo de registro es `PTR`
- La delegación sigue la **jerarquía de direcciones**: `arpa` → `in-addr.arpa` → `172` → `22` …

Zonas inversas y `dig -x`

NOMBRE DE LA ZONA INVERSA

Se invierte la **parte de red** según la máscara:

RED	ZONA INVERSA
`172.22.0.0/16`	`22.172.in-addr.arpa`
`192.168.0.0/24`	`0.168.192.in-addr.arpa`

- Dentro de la zona, el nombre de cada equipo es la **parte de host** invertida
- Hay **un** `PTR` **por cada** `A` de la zona directa

CONSULTA CON `DIG -X`

```
$ dig -x 8.8.8.8
;8.8.8.8.in-addr.arpa.   IN   PTR
8.8.8.8.in-addr.arpa. 74068 IN PTR dns.google.
```

- `-x` **invierte la IP** y pide el `PTR` por ti
- Equivale a `dig PTR 8.8.8.8.in-addr.arpa`

ℹ️ La zona inversa la gestiona quien tiene **delegado el bloque de IPs** (el proveedor), no quien tiene el dominio. Para IPv6 se usa `ip6.arpa`.

Autoridad sobre una zona

- El **servidor DNS** que guarda el **fichero de una zona** es su **servidor con autoridad**
- En el propio fichero de zona se declara con un **registro NS**
- Conviene tener **varios** servidores con autoridad por zona, por disponibilidad:

SERVIDOR MAESTRO

- Es donde se **edita** la zona
- Su fichero de zona es la **fuente de verdad**

SERVIDOR ESCLAVO

- Tiene una **copia** de la zona
- La obtiene del maestro por **transferencia de zona** (TCP) y se mantiene sincronizada
- Responde con **autoridad** igual que el maestro

ℹ️ Los servidores con autoridad sobre el **dominio raíz** se llaman **root servers**: **13 conjuntos** (A–M), cada uno replicado con

Delegación de dominios

> *DNS es una **jerarquía de zonas**: cada zona puede **delegar** una parte de su espacio de nombres en otro servidor.*

- La zona `org` **delega** `gonzalonazareno.org` en sus servidores con autoridad
- Para delegar, la zona **padre** guarda registros **NS** del subdominio: *«para `sub.dominio.org`, pregunta a este servidor»*
- Quien recibe la delegación es **autoridad** de esa parte y puede delegar a su vez
- Así nadie necesita conocer todos los nombres: **cada zona gestiona lo suyo**

⚠️ **Glue records:** si el servidor con autoridad está *dentro* del propio subdominio delegado (`ns.sub.dominio.org`), la zona padre debe guardar también su **IP**. Si no, habría un círculo: para preguntar al servidor necesito su IP, y para su IP debo preguntarle a él.

Tipos de registros más usados

REGISTRO	PARA QUÉ SIRVE
SOA	*Start of Authority* — metainformación de la zona (serie, tiempos de sincronización y caché)
NS	Servidor con autoridad sobre la zona (y delegaciones)
A	Dirección **IPv4** de un nombre
AAAA	Dirección **IPv6** de un nombre
CNAME	**Alias** — otro nombre que apunta al «real»
MX	Servidor de **correo** del dominio
PTR	Resolución **inversa** (IP → nombre)
TXT	Texto libre (SPF, DKIM, verificación…)

02

Resolución y recursión

Cómo se resuelve un nombre

¿Quién pregunta a quién?

Cuando un equipo necesita resolver un nombre:

1. Comprueba la **resolución estática** (`/etc/hosts`, según `nsswitch.conf`)
2. Si no lo encuentra, pregunta al **servidor DNS** configurado en `/etc/resolv.conf`
3. Ese servidor, si no sabe la respuesta, **busca por él** en la jerarquía DNS

CONSULTA RECURSIVA

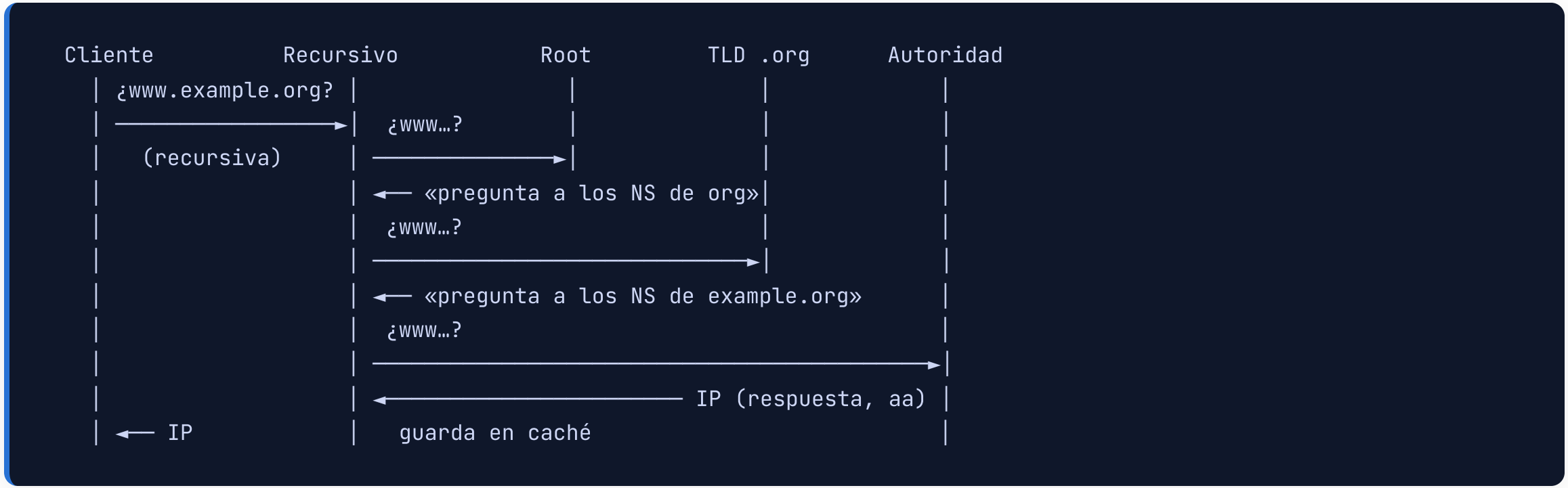
- La hace el **cliente** (*stub resolver*) a su servidor DNS
- Significa: *«no me des pistas, **devuélveme la respuesta final** o un error»*
- El servidor asume **todo el trabajo**

CONSULTA ITERATIVA

- La hace el **servidor recursivo** a los servidores de la jerarquía
- Cada uno responde *«no lo sé, **pregunta a este otro**»* (una **referencia**)
- El servidor recursivo **sigue las pistas** hasta la autoridad

Resolución paso a paso

Un cliente pregunta `www.example.org` a su servidor recursivo (caché vacía):



ℹ️ El cliente hace **una sola pregunta**. Con `dig +trace` es `dig` quien hace el papel del recursivo y te enseña cada salto.

Qué responde cada servidor

Los servidores intermedios **no dan la respuesta**: dan una **referencia**.

SECCIÓN	EN UNA REFERENCIA	EN LA RESPUESTA FINAL
`ANSWER`	**vacía**	el registro pedido (`A`, `MX`...)
`AUTHORITY`	registros **NS** del siguiente nivel	normalmente vacía
`ADDITIONAL`	**IPs** de esos NS (*glue*)	datos extra (EDNS)

- El **root** responde con los NS del **TLD** (`org`)
- El **TLD** responde con los NS del **dominio** (`example.org`)
- El servidor **con autoridad** responde con la IP y marca la respuesta como autoritativa (`aa`)

Caché y TTL

- Cada registro lleva un **TTL**: segundos que se puede **guardar en caché**
- Mientras no caduque, el servidor recursivo responde **sin preguntar a nadie**
- El TTL baja en cada consulta repetida: es la forma de ver si una respuesta viene de caché

```
$ dig www.debian.org
www.debian.org.   162   IN   A   194.177.211.216     ← quedan 162 s en caché
```

CACHÉ POSITIVA

- Se guarda la **respuesta correcta**
- Duración: el **TTL** del registro

CACHÉ NEGATIVA

- También se guarda que un nombre **no existe** (`NXDOMAIN`)
- Duración: el campo *negative TTL* del **SOA** de la zona

Tipos de servidores DNS

SERVIDOR RECURSIVO

- Resuelve **cualquier consulta** preguntando él mismo a la jerarquía
- Mantiene **caché** de respuestas
- Es el típico de los proveedores y de las redes locales

SERVIDOR FORWARD

- **Reenvía** las consultas a otro servidor recursivo
- No resuelve por sí mismo desde la raíz

SERVIDOR CON AUTORIDAD

- Responde **sólo** sobre las zonas que aloja
- **No hace recursión** con clientes: no resuelve lo que no es suyo
- Es la fuente «oficial» de los registros del dominio

SERVIDOR CACHÉ

- Sólo guarda respuestas previas
- Útil como capa intermedia para acelerar consultas

DNS en una red local

¿Y si el servidor DNS configurado en `resolv.conf` está en **nuestra propia red**?

- Las resoluciones se hacen **más rápidas**: lo que esté en caché no se vuelve a preguntar a Internet
- Si tenemos **servicios internos** con nombres propios, podemos crear una **zona local**: el servidor DNS tendrá autoridad sobre ella y resolverá esos nombres a **IPs privadas**
- El servidor **DHCP** de la red repartirá ese DNS local a los clientes
- Un mismo servidor puede ser **autoridad** de nuestras zonas y **recursivo** para el resto de nombres

ℹ️ Esto permite tener nombres como `impresora.lan` o `servidor.lan` sin depender de DNS externos ni tocar `/etc/hosts` en cada equipo.

03

Anatomía de una respuesta

`dig`, cabecera y bits de control

La herramienta `dig`

`dig` hace **consultas DNS** desde la línea de comandos y muestra **toda** la respuesta del servidor, no sólo la IP.

```
dig [tipo de registro] [@servidor DNS] consulta_DNS
```

- **Tipo de registro** por defecto: `A`. **Servidor** por defecto: el de `/etc/resolv.conf`
- `-x <IP>` para la resolución **inversa**
- `+short` respuesta resumida · `+trace` sigue la jerarquía desde la raíz · `+norec` consulta **sin recursión**
- Paquete Debian: **`dnsutils`** (`sudo apt install dnsutils`)

Salida completa de `dig`

```
$ dig mx debian.org

; <<>> DiG 9.20.30 <<>> mx debian.org                          ← ① comando
;; Got answer:
;; ⟶»HEADER«⟵ opcode: QUERY, status: NOERROR, id: 16139   ← ② cabecera
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:                                        ← ③ EDNS
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:                                         ← ④ pregunta
;debian.org.                    IN      MX

;; ANSWER SECTION:                                           ← ⑤ respuesta
debian.org.             28800   IN      MX      0 mailly.debian.org.
debian.org.             28800   IN      MX      0 muffat.debian.org.
debian.org.             28800   IN      MX      0 mitropoulos.debian.org.

;; Query time: 27 msec                                       ← ⑥ pie
;; SERVER: 100.100.100.100#53(100.100.100.100) (UDP)
;; WHEN: Mon Nov 09 14:07:21 CET 2026
```

② La cabecera (`HEADER`)

```
;; ⟶»HEADER«⟵ opcode: QUERY, status: NOERROR, id: 16139
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1
```

CAMPO	SIGNIFICADO
`opcode`	Tipo de operación: `QUERY` es la consulta normal
`status`	**Resultado**: `NOERROR`, `NXDOMAIN`, `SERVFAIL`, `REFUSED`
`id`	Número que **empareja** cada respuesta con su consulta (también dificulta la suplantación)
`flags`	**Bits de control** del mensaje: `qr rd ra ad`... (se detallan más adelante)
`QUERY`	Nº de preguntas (siempre **1**)
`ANSWER` / `AUTHORITY` / `ADDITIONAL`	Nº de **registros** que hay en cada sección

③ y ④ `OPT` y `QUESTION`

`OPT PSEUDOSECTION`

```
; EDNS: version: 0, flags:; udp: 1232
```

- **EDNS**: extensión del protocolo
- `udp: 1232` : tamaño máximo de respuesta UDP que acepta el cliente
- `flags: do` aparece al usar `+dnssec` (*DNSSEC OK*: «quiero las firmas»)
- No es un registro real: es **información del mensaje**

`QUESTION SECTION`

```
;debian.org.      IN    MX
```

- **Qué se ha preguntado**: nombre, clase y tipo
- `IN` es la clase *Internet* (la única que se usa)
- Va con `;` al principio: es un **comentario**, no un registro

⑤ ANSWER SECTION

Es **la respuesta**: una línea por **registro** (*resource record*), con cinco campos:

```
debian.org.      28800    IN    MX    0 mailly.debian.org.
   ①              ②       ③     ④            ⑤
```

	CAMPO	SIGNIFICADO
①	**Nombre**	Nombre al que se refiere el registro (FQDN, acaba en `.`)
②	**TTL**	Segundos que se puede **guardar en caché** (baja en cada consulta repetida)
③	**Clase**	`IN` (Internet)
④	**Tipo**	`A`, `AAAA`, `NS`, `MX`, `CNAME`, `PTR`, `TXT`…
⑤	**Datos**	El contenido: IP, nombre, prioridad + servidor… según el tipo

ℹ️ Si hay un `CNAME` verás **dos registros**: el alias y, a continuación, el `A` del nombre real.

`AUTHORITY` y `ADDITIONAL`

`AUTHORITY SECTION`

- Registros **NS** de los servidores con autoridad
- En una **referencia**: a quién preguntar después
- Si el nombre **no existe**: el **SOA** de la zona (con el TTL de la caché negativa)
- Con `dig ns dominio` la respuesta está en `ANSWER`; `AUTHORITY` aparece cuando es información de apoyo

`ADDITIONAL SECTION`

- Datos **extra** que ahorran otra consulta
- Típico: las **IPs** (registros `A` / `AAAA`) de los servidores de las secciones anteriores (*glue*)
- Ejemplo: al pedir los `MX`, el servidor puede añadir la IP de cada servidor de correo

⑥ El pie: servidor, tiempo y tamaño

```
;; Query time: 27 msec
;; SERVER: 100.100.100.100#53(100.100.100.100) (UDP)
;; WHEN: Mon Nov 09 14:07:21 CET 2026
;; MSG SIZE  rcvd: 113
```

LÍNEA	SIGNIFICADO
`Query time`	Tiempo hasta recibir la respuesta. **Primera** consulta: varios saltos (decenas o cientos de ms). **Repetida**: viene de **caché** (≈ 0-1 ms)
`SERVER`	**Qué servidor ha contestado**: IP, puerto `53` y protocolo (**UDP** o TCP). Por defecto el de `resolv.conf`; se cambia con `@servidor`
`WHEN`	Fecha y hora de la consulta
`MSG SIZE rcvd`	Tamaño en **bytes** de la respuesta recibida

Los bits de la cabecera DNS

La cabecera de cada mensaje DNS lleva **bits de control** (*flags*). Los de `dig` se ven en la línea `flags:`.

BIT	NOMBRE	SIGNIFICADO
`qr`	*Query/Response*	`0` = consulta, `1` = **respuesta**
`aa`	*Authoritative Answer*	la respuesta viene de un servidor **con autoridad**
`rd`	*Recursion Desired*	el cliente **pide** recursión
`ra`	*Recursion Available*	el servidor **ofrece** recursión
`tc`	*Truncated*	respuesta **truncada**: repetir por **TCP**
`ad`	*Authenticated Data*	datos **validados con DNSSEC**
`cd`	*Checking Disabled*	el cliente pide **no validar** DNSSEC

Recursión en los bits: `rd` y `ra`

La recursión es una **negociación** entre dos bits:

- El cliente envía `rd` (*«quiero recursión»*)
- El servidor contesta `ra` si **está dispuesto** a hacerla

SERVIDOR RECURSIVO

```
;; flags: qr rd ra
```

- Recibe `rd`, responde con `ra`
- Sin `aa`, salvo que el nombre sea de una zona suya

SERVIDOR CON AUTORIDAD

```
;; flags: qr aa rd
```

- Responde con `aa` y **sin** `ra`
- Si preguntas por un nombre ajeno: referencia o `REFUSED`

`aa` y `tc`: autoridad y tamaño

`AA` — RESPUESTA AUTORITATIVA

- La respuesta viene **de la fuente**: el servidor tiene la zona
- Una respuesta **desde caché** nunca lleva `aa`
- Sirve para distinguir si el dato es **oficial** o puede estar **desfasado** por la caché

`TC` — RESPUESTA TRUNCADA

- En **UDP** una respuesta tiene un tamaño máximo (con EDNS suele ser ~1232 bytes)
- Si no cabe, el servidor la corta y activa `tc`
- El cliente **repite la consulta por TCP** (`dig` lo indica: *Truncated, retrying in TCP mode*)

Códigos de estado (`status`)

ESTADO	SIGNIFICADO
`NOERROR`	consulta correcta (puede venir con la sección `ANSWER` vacía si el nombre existe pero no tiene ese tipo de registro)
`NXDOMAIN`	el nombre **no existe**
`SERVFAIL`	el servidor **no ha podido resolver** (servidor autoritativo caído, o **fallo de validación DNSSEC**)
`REFUSED`	el servidor **se niega** a responder (p. ej. no permite recursión o consultas desde tu red)

ℹ️ Son los errores que verás al montar tu propio servidor: `REFUSED` suele ser un control de acceso mal configurado y `SERVFAIL` una zona con errores.

04

DNSSEC

Autenticar las respuestas DNS

El problema: DNS no se autentica

- Una respuesta DNS **no demuestra** que venga de quien dice
- Un atacante puede **suplantarla** y llevarte a otra IP: **envenenamiento de caché** (*cache poisoning*)
- La respuesta falsa queda guardada en el **recursivo** y la reciben **todos sus clientes** hasta que caduque el TTL

⚠️ DNS no cifra ni firma. **DNSSEC** (*DNS Security Extensions*) añade **autenticidad e integridad**: permite comprobar que la respuesta la ha generado el dueño de la zona y que no se ha modificado. **No cifra** (no da confidencialidad).

Cómo funciona: firmas y claves

El dueño de la zona **firma** sus registros con **criptografía de clave pública**:

REGISTRO	CONTENIDO
DNSKEY	La **clave pública** de la zona
RRSIG	La **firma** de cada conjunto de registros (por ejemplo, los `A` de un nombre)
DS	*Delegation Signer*: **huella** de la clave de la zona hija, guardada en la zona **padre**
NSEC / NSEC3	**Prueba de inexistencia** firmada: demuestra que un nombre *no existe*

- Normalmente hay dos claves: **KSK** (firma las claves, se identifica en el DS) y **ZSK** (firma los registros)
- Un **resolver validador** recalcula la firma con la DNSKEY y comprueba que coincide

La cadena de confianza

¿Por qué confiar en la clave de `debian.org`? Porque está **avalada por el padre**, y así hasta la raíz:

```
.  (raíz)      clave pública conocida de antemano (trust anchor)
 │  DS de org     firmado por la raíz
 ▼
org            DNSKEY de org, avalada por el DS
 │  DS de debian.org   firmado por org
 ▼
debian.org     DNSKEY de debian.org  →  RRSIG de sus registros
```

- El resolver **sólo necesita conocer la clave de la raíz**: el resto se valida **encadenando** DS → DNSKEY → RRSIG
- Si **toda la cadena** es válida, devuelve la respuesta con el bit `ad` activo
- Si una firma **falla**, devuelve `SERVFAIL`: es mejor no responder que dar un dato falso

DNSSEC con `dig`

```
$ dig +dnssec debian.org
;; flags: qr rd ra ad;                    ← ad: validado
debian.org.   172  IN  A      151.101.2.132
debian.org.   172  IN  RRSIG A 8 2 300 20261211013633 20261101011742 29026 debian.org. NXTIX…

$ dig DS debian.org +short
40756 8 2 BBE42151B1A41EFC1F7E6D74D86E601D55051D2B9EB99B1CD0CEAA6E2A5618F5

$ dig DNSKEY debian.org +short
256 3 8 AwEAAaFKOvqD3ZPE/…        ← 256 = ZSK
257 3 8 AwEAAcOt7qF88Cx7…         ← 257 = KSK
```

- **`+dnssec`** pide los registros `RRSIG` junto con la respuesta
- El `RRSIG` indica algoritmo, validez (fechas de inicio y caducidad) y el *key tag* de la clave que firma

Un fallo de validación

`dnssec-failed.org` es un dominio con la firma **rota a propósito** para probar los validadores:

```
$ dig dnssec-failed.org
;; ⟶»HEADER«⟵ opcode: QUERY, status: SERVFAIL
;; flags: qr rd ra;            ← sin ad
```

```
$ dig +cd dnssec-failed.org     ← cd: «no valides»
;; ⟶»HEADER«⟵ opcode: QUERY, status: NOERROR
```

ℹ️ El nombre existe y el servidor tiene la IP, pero el **resolver validador la descarta** porque no puede comprobar la firma. Con `+cd` se desactiva la validación y sí responde.

La firma de la zona

Firmar una zona consiste en **generar las claves** y **publicar** DNSKEY, RRSIG y NSEC en ella, **renovando las firmas** antes de que caduquen.

1. Generar las claves **KSK** y **ZSK**
2. Firmar todos los conjuntos de registros → aparecen los **RRSIG**
3. Publicar el **DS** (huella de la KSK) en la **zona padre**: sin DS, **no hay cadena de confianza**
4. Mantener la zona: las firmas **caducan** y las claves se **rotan**

> ℹ️ En `bind9` moderno la firma y la rotación son **automáticas**: la zona se asocia a una *política de firma* (`dnssec-policy`) y bind gestiona claves y firmas. Se verá en los ejercicios.

¿Quién firma? Nuestro escenario

Tenemos **dos servidores DNS** con papeles distintos:

DNS EXTERNO (PROVEEDOR)

- Aloja la **zona pública** del dominio
- El proveedor suele ofrecer **firmar con un clic**
- Hay que publicar el **DS** en el **registrador** del dominio, que lo sube a la zona padre
- Si el **registrador y el proveedor** son distintos, el DS se copia a mano

NUESTRO BIND (LOCAL)

- Autoridad de las zonas **internas** (`.lan`, vista interna…)
- Como **recursivo**, **valida** las respuestas de Internet (`dnssec-validation`)
- Firmar nuestra zona sólo tiene sentido si hay **DS en el padre**: en zonas internas o delegadas a mano **no hay cadena de confianza**

DNSSEC en nuestras vistas

	VISTA PÚBLICA (INTERNET)	VISTA INTERNA (INTRANET)
Autoridad	**Proveedor** (CDMON)	**Nuestro bind**
Firma	La puede firmar el proveedor + **DS** en el registrador	Normalmente **sin firmar**
Validación	La hacen los **recursivos** de los clientes	La hace **nuestro recursivo**

⚠️ Si el nombre **interno** existe también en Internet pero con otra IP (*split-horizon*), un validador que lo comprobara con DNSSEC público **fallaría**. Por eso nuestro recursivo no valida las zonas sobre las que tiene autoridad.

ℹ️ Hoy `gonzalonazareno.org` **no está firmado**: no hay registro `DS` en `org`. Es lo habitual en la mayoría de dominios.

05

DNS en el Gonzalo Nazareno

Un mismo dominio, dos vistas

Dos zonas para `gonzalonazareno.org`

El dominio `gonzalonazareno.org` tiene **dos zonas de resolución directa** distintas:

VISTA PÚBLICA (INTERNET)

- Servidores DNS con autoridad alojados en la empresa **CDMON**
- `openstack.gonzalonazareno.org` resuelve a la **IP pública**: `5.196.224.198`

VISTA INTERNA (INTRANET)

- Servidor DNS con autoridad: `macaco` (`172.22.0.1`)
- `openstack.gonzalonazareno.org` resuelve a `172.22.0.3` (**simio**, proxy inverso)

ℹ️ Es un caso clásico de **split-horizon DNS**: la respuesta depende de desde dónde se pregunte.

Desde casa (vista pública)

```
$ dig ns gonzalonazareno.org

;; ANSWER SECTION:
gonzalonazareno.org.    21599    IN    NS    ns2.cdmon.net.
gonzalonazareno.org.    21599    IN    NS    ns4.cdmondns-01.org.
gonzalonazareno.org.    21599    IN    NS    ns1.cdmon.net.
gonzalonazareno.org.    21599    IN    NS    ns5.cdmondns-01.com.
gonzalonazareno.org.    21599    IN    NS    ns3.cdmon.net.

$ dig openstack.gonzalonazareno.org

;; ANSWER SECTION:
openstack.gonzalonazareno.org. 0 IN  CNAME  macaco.gonzalonazareno.org.
macaco.gonzalonazareno.org.    0 IN  A      5.196.224.198
```

Desde el aula (vista interna)

```
$ dig ns gonzalonazareno.org

;; ANSWER SECTION:
gonzalonazareno.org.    86400    IN    NS    dns.gonzalonazareno.org.

$ dig openstack.gonzalonazareno.org

;; ANSWER SECTION:
openstack.gonzalonazareno.org. 86400 IN  CNAME  simio.gonzalonazareno.org.
simio.gonzalonazareno.org.     86400 IN  A      172.22.0.3
```

ℹ️ El mismo nombre devuelve **respuestas distintas** según el servidor DNS al que se pregunte: el del aula tiene su propia zona con autoridad.

SRI · UNIDAD 3 — PROTOCOLO DNS

¡Gracias!

DNS

📧 José Domingo Muñoz 🏫 IES Gonzalo Nazareno · Dos Hermanas 📚 SRI